

PROJECT SPECIFICATION DOCUMENT

Hyper-Local Disaster Relief & SOS Coordinator

A community-driven emergency coordination platform built as a pure front-end web application, deployable on GitHub Pages.

Course Code	23CSE113 — UID Project 2
Group	Group IX
Deployment Target	GitHub Pages (Static Hosting)
Architecture	Pure Front-End — HTML5, CSS, JavaScript
Document Version	2.0 Final

GROUP MEMBERS		
AM. SC. U4CSE25346	S.V. Karthik	— Module 1 — SOS and help forms
AM. SC. U4CSE25345	C. Saket	— Module 2 — UI building(main HTML and CSS)
AM. SC. U4CSE25340	P. Sputnik	— Module 3 — Radar Map
AM. SC. U4CSE25337	N.H. Swapnil	— Module 4 — Live feed
AM. SC. U4CSE25334	M. Yaswanth	— Module 5 — Admin and PWA

1.0 PROJECT OVERVIEW

1.1 Purpose & Problem Statement

Natural and urban disasters — flooding, extreme weather events, power-grid failures — overwhelm centralised emergency services. Citizens are left unable to reach overloaded hotlines and unaware of nearby resources or fellow citizens who need help.

The Hyper-Local Disaster Relief & SOS Coordinator addresses this gap by providing a lightweight, browser-based platform where members of a community can post SOS requests, offer supplies or shelter, and visually locate one another on an interactive map — all without relying on a dedicated server or mobile app installation.

1.2 Key Objectives

- Bypass jammed emergency lines through direct peer-to-peer coordination.
- Give rescuers and affected citizens a shared real-time situational picture.
- Operate on low-bandwidth (2G/3G) connections during infrastructure degradation.
- Require zero installation — accessible from any modern smartphone browser.

1.3 Technology Stack

Category	Technology	Purpose
Structure	HTML5 (Semantic)	Accessibility & screen-reader support
Presentation	CSS3 (Flexbox / Grid)	Responsive, mobile-first layout
Behaviour	Vanilla JavaScript ES6+	Interactivity, state management, API integration
Data Storage	LocalStorage / SessionStorage	Simulated persistence without a server
Mapping	Leaflet.js (CDN)	Interactive emergency map with pins
Geolocation	HTML5 Geolocation API	Auto-centres map on user's position
Data Viz	Chart.js (CDN)	Admin dashboard charts and graphs

2.0 SYSTEM ARCHITECTURE & MODULE BREAKDOWN

The application is decomposed into five independent front-end modules, each owned by one team member. This ensures parallel development with clearly bounded responsibilities and minimal merge conflicts.

MODULE 1	Homepage & Global UI Assigned to: <i>S.V. Karthik (AM.SC.U4CSE25346)</i> Establishes the visual identity of the entire platform, builds the entry-point user journey, and defines the shared CSS design system consumed by all other modules.
---------------------------	---

Functional Requirements

- Hero section with high-impact visuals and two primary CTAs: "I Need Help" and "I Can Help".
- Sticky responsive navigation bar with a JavaScript-toggled hamburger menu for mobile viewports.
- Global CSS design-token system via :root custom properties (colours, typography, spacing).

Key Technical Details

- **Responsive breakpoints** — CSS @media (max-width: 768px) shifts multi-column layouts to single-column stacks.
- **Semantic HTML5** — <header>, <main>, <footer>, <section>, <nav> — required for screen-reader compliance.
- **CSS custom properties** — --color-alert, --color-safe, --font-primary defined in :root and reused site-wide.

MODULE 2	User Intake & Form Validation Assigned to: <i>C. Saket (AM.SC.U4CSE25345)</i> Covers the informational pages of the site and enforces client-side data integrity before any user submission is processed or stored.
---------------------------	---

Functional Requirements

- About page: platform mission, usage guidelines, and team biographies.
- Contact / Feedback page: form for reporting bugs, leaving feedback, or contacting moderators.
- Inline validation with descriptive, field-level error messages — no page reload on failure.

Key Technical Details

- **Regex validation** — Phone numbers validated against `/^[0-9]{10}$/` , emails against presence of `'@'` and a domain suffix.
- **DOM feedback** — JavaScript injects red error text beneath failing fields using `createElement` and `classList`.
- **Submission guard** — `event.preventDefault()` blocks form resubmission until all fields pass validation.

MODULE

3

Live Map Dashboard

Assigned to: *P. Sputnik (AM.SC.U4CSE25340)*

The most technically complex module: a full-screen interactive map showing real-time emergency pins, auto-centred on the user's physical location.

Functional Requirements

- Full-width Leaflet.js map embedded as the primary dashboard view.
- Browser Geolocation API prompt — map auto-centres on granted coordinates.
- Red pins for active SOS requests; green pins for available volunteer resources.
- Popup detail cards on pin click, showing emergency type, time posted, and contact info.

Key Technical Details

- **Pin rendering loop** — JSON array of coordinates parsed and iterated; `L.marker([lat, lng]).addTo(map)` called per entry.
- **Popups** — `marker.bindPopup('<b>Type:</b> Flooding...')` attaches contextual HTML to each pin.
- **Fallback** — If geolocation is denied, map defaults to a pre-configured city-centre view.

MODULE

4

SOS Request & Resource Engine

Assigned to: *N.H. Swapnil (AM.SC.U4CSE25337)*

The logic heart of the application: captures SOS and resource offer submissions, persists them in `LocalStorage`, and renders a live community feed without any page reload.

Functional Requirements

- "Request Help" form: emergency type, number of people affected, location description, contact.
- "Offer Resources" form: resource type (food/water/shelter/medic), quantity, duration available.
- Live community feed auto-generated from stored data — cards appear immediately on submission.
- Individual card deletion to mark requests as resolved.

Key Technical Details

- **Serialisation** — `JSON.stringify()` converts submission objects; `localStorage.setItem('emergencies', data)` persists them.
- **Retrieval** — `localStorage.getItem()` + `JSON.parse()` hydrates the feed on every page load.
- **Dynamic DOM** — `document.createElement()` + template literals build `<div>` cards injected into the feed container.

MODULE

5

Admin Dashboard & Analytics

Assigned to: *M. Yaswanth (AM.SC.U4CSE25334)*

A simulated moderator view providing platform-wide statistics, data visualisations, and a live emergency broadcast ticker — fulfilling the additional page requirement.

Functional Requirements

- Dark-mode dashboard layout styled for simulated administrative authority.
- Bar chart: "Resources Needed vs. Resources Available" drawn from Module 4 storage data.
- Doughnut chart: "Emergency Types Reported" as a percentage breakdown.
- Scrolling ticker banner cycling safety protocols every 5 seconds.

Key Technical Details

- **Chart.js** — Loaded via CDN; charts instantiated with `new Chart(ctx, { type, data, options })`.
- **Live ticker** — `setInterval(() => { /* update text */ }, 5000)` cycles through a broadcast message array.
- **Data source** — Charts read directly from `localStorage` populated by Module 4, ensuring live metrics.

3.0 NON-FUNCTIONAL REQUIREMENTS

Requirement	Target / Constraint	Implementation Approach
Usability	Zero training required	Large tap targets (≥ 44 px), high-contrast colours, single-tap primary actions for stressed users operating in low-light.
Performance	Load in < 3 s on 3G	Image compression (WebP / optimised JPEG), minified CSS/JS, deferred non-critical scripts, single CDN for Leaflet & Chart.js.
Accessibility	WCAG 2.1 Level AA	Semantic HTML5 elements, aria-label on icon buttons, sufficient colour contrast ratio ($\geq 4.5:1$), keyboard navigability.
Compatibility	All modern browsers	Chrome for Android, Safari for iOS, Firefox, and Chrome Desktop. CSS Grid with Flexbox fallbacks; no CSS features behind flags.
Resilience	No server dependency	Full functionality from a static file-server (GitHub Pages). LocalStorage fallback alert if storage quota is exceeded.

4.0 TESTING STRATEGY

Prior to deployment on GitHub Pages, the team will execute a structured test plan across three categories:

4.1 Responsive Layout Testing

- Tool: Google Chrome DevTools device emulation.
- Viewports: iPhone SE (375 px), iPad (768 px), standard desktop (1440 px), wide desktop (1920 px).
- Pass criteria: no horizontal scroll, no overlapping elements, all buttons remain tappable.

4.2 LocalStorage State Testing

- Manual scenario: submit SOS and resource-offer forms, fully close and reopen the browser.
- Pass criteria: all entries reappear in the community feed without re-entry by the user.
- Edge case: fill LocalStorage to quota — confirm graceful error alert is shown.

4.3 Integration & Link Verification

- Audit all internal navigation links, anchor tags, and CDN script loads (Leaflet.js, Chart.js).
- Confirm zero JavaScript console errors on each page under Chrome and Safari.
- Verify Geolocation API permission prompt fires correctly and map renders on both grant and deny.

5.0 PAGE & FILE STRUCTURE

The repository will be organised as follows to ensure clean separation of concerns and straightforward GitHub Pages deployment:

```
/ (repository root)
├── index.html           ← Module 1: Homepage
├── about.html          ← Module 2: About & Mission
├── contact.html        ← Module 2: Contact / Feedback
├── map.html            ← Module 3: Live Map Dashboard
├── sos.html            ← Module 4: SOS & Resource Forms
├── feed.html           ← Module 4: Community Feed
├── admin.html          ← Module 5: Admin Dashboard
├── css/
│   ├── global.css      ← CSS variables, resets, typography
│   └── [page].css      ← Per-module stylesheets
├── js/
│   ├── nav.js          ← Hamburger menu (shared)
│   ├── validation.js   ← Form validation helpers
│   ├── map.js          ← Leaflet integration
│   ├── storage.js       ← LocalStorage read/write helpers
│   └── admin.js        ← Chart.js charts & ticker
└── assets/
    └── img/            ← Compressed WebP images
```

6.0 USER DIAGRAM

